EXACT ARITHMETIC AT SMALL SALEM NUMBERS
NEW PARRY CERTIFICATES IN DEGREE SIX, RESULTANT SQUARES, COINCIDENCE PRIMES, AND TWO TRANSFER OBSTRUCTIONS

KEIO

Abstract. We report certified computations and short proofs around two classical questions on Salem numbers: Schmidt's question whether every Salem number is a Parry number, and Lehmer's problem.

(1) We build a certified engine for the orbit of 1 under the β-transformation. States are integer vectors in $\mathbb{Z}[\beta]$; each digit is decided by floating-point evaluation under an a priori error bound, with a quadruple-precision fallback and a final exact fallback (a zero test combined with a norm inequality); cycles are found by Brent's algorithm together with distinguished points. Of the fifteen resolved entries of Stockton's census of sextic Salem numbers that we recomputed, the engine reproduces thirteen exactly; for the other two its preperiod is larger by one, and an independent check confirms our values. Applied to the cases of trace 11 to 15 left open by Boyd and Stockton, it resolves five of the twenty-seven cases we processed; for instance, the Salem number of $x^6 - 14x^5 - 30x^4 - 41x^3 - 30x^2 - 14x + 1$ has an eventually periodic expansion of 1 with preperiod 655 666 845 and period 370 445 447. Every new certificate was re-verified by an independent program. For the other attacked cases we raise the rigorous lower bound on preperiod plus period by factors between 3.1 and 9.8.

(2) For a monic reciprocal polynomial $P(x) = x^m Q(x + x^{-1})$ we prove $\mathrm{Res}(P(x), P(x^p)) = \mathrm{Res}(Q, Q \circ D_p)^2$, where D_p is the Dickson polynomial. Consequently Dobrowolski's resultant equals $p^{2m} A_p^2$ with $A_p = |\mathrm{N}(Q(D_p(t))/p)|$, and for primes $\ell \neq p$ one has $\ell \mid A_p$ exactly when the p-th power map sends a root of P modulo ℓ to a root. For Lehmer's polynomial $A_2 = A_3 = 1$, so Dobrowolski's p-adic inequality is an equality, while $3 \mid A_p$ for every prime $p \geq 5$ because $x^2 + 1$ divides it modulo 3. We tabulate A_p for the 23 Salem numbers below 1.3 of degree at most 20.

(3) We record two elementary obstructions that explain why two recent mechanisms do not transfer to Lehmer's problem: certificates depending continuously on the distribution of conjugates cannot detect a Lehmer gap, and capacity criteria for star-shaped sets saturate at rate $(\log 4)/d$ independently of the Mahler measure.

1. Introduction

A *Salem number* is a real algebraic integer $\beta > 1$ of degree $d \geq 4$ whose other conjugates are β^{-1} and $d - 2$ numbers of modulus one. Its minimal polynomial is reciprocal of even degree, and its Mahler measure equals β. Lehmer's number

$$\lambda = 1.17628081825991750654\ldots, \qquad L(x) = x^{10} + x^9 - x^7 - x^6 - x^5 - x^4 - x^3 + x + 1,$$

is the smallest known Salem number and the smallest known Mahler measure greater than one [13]. Lehmer's problem asks whether $M(\alpha) \geq c > 1$ for every algebraic integer α that is not a root of unity. The best general lower bound is still of Dobrowolski's form $1 + c(\log\log d/\log d)^3$ [10, 25], and computations exclude Mahler measures in $(1, \lambda)$ up to degree 44 [15]. Smyth's survey [21] describes the wider landscape.

Date: October 2026.

Provenance. This manuscript was developed in collaboration with a large language model (Claude, Anthropic). Statements, proofs and methods were arrived at interactively; the author is responsible for the content. It is a research draft, not a peer-reviewed publication. *Verifiable* here means that every quantitative claim is accompanied by a program, printed in the appendices, that recomputes it.

For $\beta > 1$ the β-transformation $T_\beta(x) = \beta x - \lfloor \beta x \rfloor$ produces the greedy expansion $d_\beta(1) = .d_1 d_2 d_3 \cdots$ of 1. The number β is a *Parry number* if $d_\beta(1)$ is eventually periodic (finite expansions included) [18, 16]. Schmidt proved that Pisot numbers are Parry numbers and asked whether the same holds for Salem numbers [19]. Boyd answered this affirmatively in degree four [5]. In degree six he computed the expansions of all 11 836 Salem numbers of trace at most 15, found eventually periodic expansions for all but 81 of them, and gave a heuristic model predicting that degree-six Salem numbers are Parry numbers with orbits that can be arbitrarily long, while in degree at least eight a positive proportion should not be Parry numbers [6]. Stockton revisited the 81 open cases [22]. He resolved 17 of them, the largest with preperiod 26 440 386 599, and recorded lower bounds for the remaining 64. Hichri studied degree eight [11], and Akiyama and Hichri proved that β^m is a Parry number for a set of exponents m of positive density [1].

Contributions. Table 1 summarizes the status of each item.

(A) A certified engine for $d_\beta(1)$ (Section 3). It reproduces thirteen of the fifteen resolved entries of Stockton's census that we recomputed exactly, and corrects the preperiod of the other two by one (Table 2).
(B) New Parry certificates in degree six (Theorem 4.1, Table 3). Of the twenty-seven open cases of trace 11 to 15 that we attacked, five are resolved, each re-verified by an independent checker.
(C) Improved rigorous lower bounds for preperiod plus period in the remaining attacked cases, by factors between 3.1 and 9.8 (Table 4).
(D) For reciprocal P, Dobrowolski's resultant is the square of a resultant of trace polynomials (Theorem 5.1); this yields a unit criterion for the equality case (Corollary 5.2).
(E) A characterization of the primes dividing the cofactor A_p (Proposition 5.3), its application to Lehmer's polynomial modulo 3 (Example 5.6), and a table of A_p (Table 5).
(F) Two transfer obstructions (Propositions 6.1 and 6.2).
(G) Independent re-verifications of known results (Section 7).

Table 1. Status of the results.

Item	Content	Status
(A)	certified engine; agreement with Stockton's entries, two preperiods corrected by one	method; reproduction; erratum
(B)	five new eventually periodic expansions in degree six	new to our knowledge
(C)	improved lower bounds for open cases	new
(D)	$\mathrm{Res}(P(x), P(x^p)) = \mathrm{Res}(Q, Q \circ D_p)^2$, unit criterion	elementary; not found stated
(E)	coincidence primes; Lehmer's polynomial modulo 3	elementary; observation
(F)	two transfer obstructions	formalization of known principles
(G)	re-verifications	reproduction

The literature search behind the word "new" covered the sources cited here through October 2026. We cannot exclude unpublished computations.

2. Preliminaries

2.1. **Reciprocal polynomials and trace polynomials.** Let $P \in \mathbb{Z}[x]$ be monic of degree $2m$ with $P(x) = x^{2m} P(1/x)$. There is a unique monic $Q \in \mathbb{Z}[y]$ of degree m with

$$x^{-m} P(x) = Q(x + x^{-1}),$$

and $P(x)=\prod_{k=1}^{m}(x^2-t_kx+1)$, where $t_1,\dots,t_m$ are the roots of Q. We call Q the *trace polynomial* of P. If P is the minimal polynomial of a Salem number β, then Q is irreducible, $t_1=\beta+\beta^{-1}>2$, and $t_2,\dots,t_m\in(-2,2)$. For the sextic polynomials

$$P_{a,b,c}(x)=x^6+ax^5+bx^4+cx^3+bx^2+ax+1$$

used by Boyd and Stockton, $Q(y)=y^3+ay^2+(b-3)y+(c-2a)$, and the trace of the Salem number $\beta_{a,b,c}$ is $-a$.

2.2. **Dickson polynomials.** Put $D_0=2$, $D_1=y$ and $D_{k+1}=yD_k-D_{k-1}$. Then $D_k\in\mathbb{Z}[y]$ is monic of degree $k\ge1$ and $D_k(x+x^{-1})=x^k+x^{-k}$ [14]. For a prime p,

$$D_p(y)=\sum_{j=0}^{\lfloor p/2\rfloor}\frac{p}{p-j}\binom{p-j}{j}(-1)^jy^{p-2j}\equiv y^p\pmod{p\mathbb{Z}[y]},$$

because for $1\le j\le\lfloor p/2\rfloor$ the coefficient $N_j=\frac{p}{p-j}\binom{p-j}{j}$ is an integer [14] with $(p-j)N_j=p\binom{p-j}{j}$, and $\gcd(p,p-j)=1$ forces $p\mid N_j$.

2.3. **Beta-expansions.** Fix a Salem number β. Put $r_0=1$ and, for $n\ge1$,

$$d_n=\lfloor\beta r_{n-1}\rfloor,\qquad r_n=\beta r_{n-1}-d_n.$$

Then $d_\beta(1)=.d_1d_2\cdots$, $r_n=T_\beta^n(1)$ for $n\ge1$, and $r_n=\sum_{j\ge1}d_{n+j}\beta^{-j}$ for all $n\ge0$. Thus the digits determine the states and conversely, so $d_\beta(1)$ is eventually periodic if and only if (r_n) is. In that case we write $\mu\ge0$ and $\lambda\ge1$ for the least integers with $r_\mu=r_{\mu+\lambda}$; they are the preperiod and the period of $d_\beta(1)$ (Stockton's m and p). Since β is an algebraic integer, $r_n\in\mathbb{Z}[\beta]$ for all n.

3. A certified engine for the orbit of one

3.1. **Integer states.** Let $P(x)=x^d+c_{d-1}x^{d-1}+\cdots+c_0$ be the minimal polynomial of β. We represent $r=\sum_{i<d}r_i\beta^i\in\mathbb{Z}[\beta]$ by $(r_0,\dots,r_{d-1})\in\mathbb{Z}^d$. Multiplication by β sends this vector to y with

$$y_0=-r_{d-1}c_0,\qquad y_i=r_{i-1}-r_{d-1}c_i\quad(1\le i\le d-1),$$

and the next state is $y-k\,e_0$ with $k=\lfloor y(\beta)\rfloor$, where $y(\beta)=\sum_i y_i\beta^i$. Since $1,\beta,\dots,\beta^{d-1}$ is a basis, equality of states is equality of integer vectors. The only non-exact step is the computation of k.

3.2. **Deciding a digit.**

Lemma 3.1 (norm gap)**.** *Let β be an algebraic integer of degree d whose conjugates other than β lie in the closed unit disc. If $z=\sum_{i<d}z_i\beta^i$ with $z_i\in\mathbb{Z}$ is nonzero, then*

$$|z|\;\ge\;\Big(\sum_{i<d}|z_i|\Big)^{-(d-1)}.$$

Proof. The norm of z is a nonzero rational integer, so $|z|\ge\prod_{\sigma\ne\mathrm{id}}|\sigma(z)|^{-1}$. For every embedding $\sigma\ne\mathrm{id}$ we have $|\sigma(z)|\le\sum|z_i||\sigma(\beta)|^i\le\sum|z_i|$, and $\sum|z_i|\ge1$. □

For a vector y the engine evaluates $y(\beta)$ by Horner's rule in three stages, each with an a priori bound E on the absolute error.

(1) Double precision with fused multiply–add: $E=8(d+2)u_{53}\,\|y\|_\infty\sum_{i<d}\beta^i$, where $u_{53}=2^{-53}$.
(2) Extended precision (64-bit significand, used only if stage 1 is inconclusive), with $u_{64}=2^{-64}$ in place of u_{53}; then quadruple precision (113-bit significand) with the bound $64(d+2)10^{-33}\sum_i|y_i|\beta^i$.

(3) Exact resolution. Let k^* be the integer nearest to the quadruple-precision value and $z = y - k^* e_0$. If $z = 0$ as a vector, then $y(\beta) = k^*$. Otherwise Lemma 3.1 gives $|z(\beta)| \geq \|z\|_1^{-(d-1)}$, and if the quadruple-precision error bound is smaller than this by a factor e^2, the sign of $z(\beta)$, and hence $\lfloor y(\beta) \rfloor \in \{k^* - 1, k^*\}$, is read off from the approximation.

A stage is conclusive when no integer lies within E of the computed value.

Proposition 3.2. *Whenever the procedure returns a digit, the digit equals* $\lfloor y(\beta) \rfloor$.

Proof. Horner's rule evaluated in floating point with unit roundoff u has absolute error at most $\gamma_{2d} \sum |y_i| \beta^i$ with $\gamma_n = nu/(1 - nu)$ [12, Sec. 5.1]. Replacing β by its floating-point value, which is computed to quadruple precision by bisection, adds at most about $d\,u \sum |y_i| \beta^i$. Both are dominated by the bounds used in stages 1 and 2, since $\sum |y_i| \beta^i \leq \|y\|_\infty \sum \beta^i$. If no integer lies within the bound of the computed value, the floor of the true value equals the floor of the computed value. Stage 3 is justified by Lemma 3.1. □

The program aborts if stage 3 is inconclusive; this never happened. In all runs reported below, stage 2 always decided, so stage 3 was never needed for a Salem number; in our tests it was reached only for the finite expansion of the golden ratio.

3.3. **Finding cycles.** The engine runs Brent's cycle-finding algorithm [8] and, in parallel, the method of distinguished points [23]. A state is *distinguished* if the low 14 bits of $r_0 \oplus (r_1 \ll 7) \oplus (r_2 \ll 13)$ vanish; distinguished states are stored in a hash table together with their indices. A repetition $r_i = r_j$ with $i < j$, found by either method, implies $i \geq \mu$ and $\lambda \mid j - i$. Walking from r_j until the first return gives λ exactly. The preperiod μ is then found by advancing two copies of the orbit, λ steps apart, from the latest stored state that is not on the cycle (or from r_0). Brent's method guarantees detection of short cycles. Distinguished points detect long cycles about $\mu + \lambda$ steps after the start, instead of up to $3(\mu + \lambda)$.

When no repetition is found within the budget, the run still yields a rigorous lower bound.

Lemma 3.3 (records)**.** *If* $\|r_{n_0}\|_\infty > \|r_n\|_\infty$ *for all* $n < n_0$*, then* $\mu + \lambda > n_0$.

Proof. If $\mu + \lambda \leq n_0$, then $n_0 - \lambda \geq \mu$ and $r_{n_0} = r_{n_0 - \lambda}$, so r_{n_0} is not a strict record. □

Stockton's bounds are of the same type, with a different function of the state [22, Sec. 6.1]. We report the index of the last strict record of $\|r_n\|_\infty$.

3.4. **An independent checker.** A second program (Appendix B) verifies a claimed pair (μ, λ) by direct iteration from r_0. It checks that

$$r_\mu = r_{\mu+\lambda}, \qquad r_{\mu+j} \neq r_\mu \ (0 < j < \lambda), \qquad r_{\mu-1} \neq r_{\mu-1+\lambda}.$$

These three facts say exactly that the preperiod is μ and the period is λ. The checker shares no cycle-finding code with the engine and uses extended precision instead of double precision in its first stage. It also prints a 64-bit FNV-style checksum of $d_1, \ldots, d_{\mu+\lambda}$ as a fingerprint of the expansion.

3.5. **Validation.** Table 2 compares the engine with the resolved entries of Stockton's tables. All periods agree, and thirteen of the preperiods agree exactly. For $(-8, -30, -44)$ and $(-9, -35, -51)$ our preperiods 2 121 493 282 and 4 075 324 465 exceed Stockton's by one. The independent checker confirms our values, and with Stockton's values it finds $r_m \neq r_{m+\lambda}$. Stockton defines m as the least $m \geq 0$ with $c_{n+m} = c_{n+m+p}$ for all $n \geq 1$, which coincides with our μ by Section 2.3, so these two entries of his tables appear to be off by one. The two largest entries, $(-10, -36, -52)$ and $(-5, -2, -11)$ with preperiods 2.3×10^{10} and 2.6×10^{10}, were not recomputed because of their cost. For small cases the engine gives $\mu = 1$ and $\lambda = 3$ for the quartic Salem number $1.72208\ldots$, $\mu = 1$ and $\lambda = 7$ for the sextic $1.40126\ldots$, $\mu = 1$ and $\lambda = 831$ for the decic $1.23039\ldots$, and $\mu = 1$ and $\lambda = 74$ for Lehmer's number (Proposition 7.1); we also obtained these values with a separate exact

implementation in Python. The engine runs at about 2 to 3×10^7 iterations per second per core on a 2.1 GHz Xeon processor.

TABLE 2. Resolved entries of Stockton's tables recomputed with the engine of Appendix A. In all rows the period λ coincides with Stockton's p. "agrees" means that also $\mu = m$. †Our preperiod exceeds Stockton's m by one; the independent checker confirms $r_\mu = r_{\mu+\lambda}$ for our value and $r_m \neq r_{m+\lambda}$ for Stockton's (Section 3).

(a, b, c)	trace	β	μ	λ	Stockton
$(-7, -29, -43)$	7	10.26180987	1 039 779	90	agrees
$(-11, -11, -26)$	11	12.09402751	1 285 570	677	agrees
$(-14, 13, -29)$	14	13.17513664	1 428 555	7 640	agrees
$(-11, -14, -28)$	11	12.32782779	1 490 333	72 458	agrees
$(-14, -36, -45)$	14	16.37473060	2 098 011	112	agrees
$(-15, -43, -62)$	15	17.64419766	11 994 574	217 750	agrees
$(-12, -47, -69)$	12	15.36446836	18 789 419	12 521	agrees
$(-13, -6, -27)$	13	13.59043902	10 913 948	13 914 931	agrees
$(-11, -35, -49)$	11	13.80584158	48 516 722	3 128 603	agrees
$(-13, -30, -41)$	13	15.16531524	137 293 807	164 656	agrees
$(-7, -28, -41)$	7	10.17506247	159 781 799	94 829	agrees
$(-13, -38, -55)$	13	15.66080010	191 196 227	16 067	agrees
$(-12, -43, -62)$	12	15.12634586	779 478 947	96 687	agrees
$(-8, -30, -44)$	8	11.08651327	2 121 493 282	188 611 456	$m = 2\,121\,493\,281$†
$(-9, -35, -51)$	9	12.22406673	4 075 324 465	425 719 617	$m = 4\,075\,324\,464$†

4. New results in degree six

4.1. **Targets.** Stockton's Table 3 lists 64 sextic Salem numbers of trace at most 15 whose expansions remained undetermined, with lower bounds on $\mu + \lambda$ between 4.8×10^8 and 9.1×10^{10} [22]. Before computing, we checked that all 64 triples define irreducible Salem polynomials. We skipped the twelve cases of trace at most 10, whose bounds (3×10^{10} to 9×10^{10}) exceed our budget. The remaining cases were processed in order of increasing bound, with budgets of 6×10^9 iterations (bound at most 1.1×10^9), 1.5×10^{10} (bound at most 3.2×10^9) and 2×10^{10} (otherwise). We processed the twenty-seven targets whose bound is at most 3.1×10^9, i.e. the first two budget groups; five were resolved and twenty-two remain undetermined. The twenty-five targets with larger bounds were not processed.

4.2. **New Parry certificates.**

Theorem 4.1. *For each triple (a, b, c) in Table 3, the Salem number $\beta_{a,b,c}$ is a Parry number: $d_\beta(1)$ is eventually periodic with the preperiod μ and period λ listed there.*

Proof. The engine of Section 3 produced each pair (μ, λ), and the independent checker confirmed the three defining relations of (μ, λ) by direct iteration. Proposition 3.2 guarantees that every digit computed by either program is correct. □

4.3. **Improved lower bounds.** For the attacked cases that remained undetermined, Table 4 lists the number of iterations performed and the resulting rigorous lower bound from Lemma 3.3.

Remark 4.2. The new periods range from 10 036 to 370 445 447, and the preperiods from 655 666 845 to 3 542 385 217. This is consistent with Boyd's heuristic picture in degree six [6]. The orbit behaves like a walk on a lattice in the four-dimensional space of unit-circle embeddings, the angle vector

TABLE 3. New Parry certificates: preperiod μ and period λ of $d_\beta(1)$ for $\beta = \beta_{a,b,c}$, the largest root of $x^6 + ax^5 + bx^4 + cx^3 + bx^2 + ax + 1$. The column "Stockton" gives his lower bound for $\mu + \lambda$ in units of 10^9 [22]; the last column is the digit checksum printed by the independent checker.

(a, b, c)	trace	β	μ	λ	Stockton	checksum
$(-11, -30, -44)$	11	13.48020048	3 058 897 164	49 029 661	3.08	`6112d7e6c8d36812`
$(-14, -30, -41)$	14	16.03750663	655 666 845	370 445 447	0.48	`062f63e277c4607e`
$(-15, -21, -39)$	15	16.42777756	1 159 826 739	10 360 541	0.57	`7897563ba6db8b4d`
$(-15, -6, -31)$	15	15.51727443	1 697 542 106	267 846	0.86	`45195b857d97e1ef`
$(-15, -54, -77)$	15	18.20721548	3 542 385 217	10 036	0.92	`bd396bdf1c186e19`

TABLE 4. Attacked cases still undetermined. Bounds are lower bounds for $\mu + \lambda$ in units of 10^9: Stockton's, and ours from Lemma 3.3 after the stated number of iterations (in units of 10^9). The last column is the ratio of the two bounds.

(a, b, c)	trace	β	Stockton	iterations	new bound	ratio
$(-15, 19, -34)$	15	13.794489	0.50	6	4.92	9.8
$(-15, -40, -58)$	15	17.485027	0.63	6	5.17	8.2
$(-15, -5, -31)$	15	15.454924	0.63	6	5.95	9.4
$(-15, -46, -66)$	15	17.800754	0.65	6	5.80	8.9
$(-15, -55, -80)$	15	18.260963	0.75	6	5.82	7.8
$(-15, 37, -51)$	15	12.312087	0.75	6	5.88	7.8
$(-15, -45, -65)$	15	17.749757	0.87	6	4.60	5.3
$(-15, -39, -57)$	15	17.432318	0.93	6	2.87	3.1
$(-14, 41, -57)$	14	10.608384	0.93	6	5.84	6.3
$(-15, -13, -33)$	15	15.948312	0.95	6	4.52	4.8
$(-15, -18, -37)$	15	16.252043	0.98	6	6.00	6.1
$(-15, -11, -33)$	15	15.829608	0.98	6	5.94	6.1
$(-15, -56, -82)$	15	18.311910	0.98	6	3.47	3.5
$(-15, -58, -85)$	15	18.410565	1.00	6	4.74	4.7
$(-14, -16, -34)$	14	15.204230	1.06	6	4.54	4.3
$(-14, -37, -54)$	14	16.456277	1.75	15	14.92	8.5
$(-14, -40, -58)$	14	16.624787	1.87	15	14.30	7.6
$(-14, -38, -55)$	14	16.511753	2.01	15	12.66	6.3
$(-14, -46, -66)$	14	16.952693	2.48	15	14.72	5.9
$(-14, -43, -62)$	14	16.790205	2.71	15	11.71	4.3
$(-14, -45, -65)$	14	16.899830	3.08	15	11.42	3.7
$(-13, -10, -29)$	13	13.875419	3.10	15	14.99	4.8

equidistributes on the torus (for the resulting limit law of β^n modulo 1 see [2]), the number of available states grows with the explored radius, and coincidences occur at a rate that makes long orbits common without making infinite ones likely. The cases of trace at most 10 remain open, among them $(a, b, c) = (-3, -1, -7)$, i.e. $x^6 - 3x^5 - x^4 - 7x^3 - x^2 - 3x + 1$ with $\beta = 3.78469\ldots$, for which Stockton's bound is $\mu + \lambda > 7.79 \times 10^{10}$.

5. Dobrowolski's resultant for reciprocal polynomials

Dobrowolski's argument [10] starts from the observation that for monic $P \in \mathbb{Z}[x]$ of degree d and a prime p, the resultant $R_p = \mathrm{Res}(P(x), P(x^p))$ is divisible by p^d. Hence $|R_p| \geq p^d$ when

$R_p \neq 0$, and all further work goes into bounding $|R_p|$ from above in terms of $M(P)$. For reciprocal polynomials the resultant has more structure.

Theorem 5.1. *Let $P \in \mathbb{Z}[x]$ be monic and reciprocal of degree $2m$ with trace polynomial Q. For every integer $n \geq 1$,*

$$\operatorname{Res}\big(P(x), P(x^n)\big) = \operatorname{Res}\big(Q(y), Q(D_n(y))\big)^2.$$

Proof. Since P and Q are monic, $\operatorname{Res}(P, G) = \prod_{P(z)=0} G(z)$ and $\operatorname{Res}(Q, H) = \prod_{Q(t)=0} H(t)$, with multiplicities. Substituting $x \mapsto x^n$ in $x^{-m}P(x) = Q(x + x^{-1})$ gives the Laurent identity $x^{-nm}P(x^n) = Q(D_n(x + x^{-1}))$. The roots of P are $z_k^{\pm 1}$, $k = 1, \ldots, m$, where $z_k + z_k^{-1} = t_k$. Hence

$$\operatorname{Res}(P(x), P(x^n)) = \prod_{k=1}^{m} P(z_k^n)P(z_k^{-n}) = \prod_{k=1}^{m} z_k^{nm} z_k^{-nm}\, Q(D_n(t_k))^2 = \operatorname{Res}(Q, Q \circ D_n)^2. \qquad \square$$

Corollary 5.2. *Let P be the minimal polynomial of a Salem number β of degree $2m$, let $t = \beta + \beta^{-1}$, $K = \mathbb{Q}(t)$ and let p be a prime. Then $Q(D_p(t)) \in p\,\mathbb{Z}[t]$, and*

$$\operatorname{Res}(P(x), P(x^p)) = p^{2m} A_p^2, \qquad A_p = \big|\, \mathrm{N}_{K/\mathbb{Q}}\big(Q(D_p(t))/p\big)\big| \in \mathbb{Z}_{\geq 1}.$$

In particular $|\operatorname{Res}(P(x), P(x^p))| = p^{2m}$, i.e. Dobrowolski's p-adic inequality is an equality, if and only if $Q(D_p(t))/p$ is a unit.

Proof. By the congruence $D_p \equiv y^p$ and the Frobenius identity $Q(y^p) \equiv Q(y)^p$ modulo $p\mathbb{Z}[y]$, we get $Q(D_p(t)) \equiv Q(t)^p = 0$ modulo $p\mathbb{Z}[t]$. Since Q is the minimal polynomial of t, $\operatorname{Res}(Q, Q \circ D_p) = \mathrm{N}_{K/\mathbb{Q}}(Q(D_p(t))) = p^m\, \mathrm{N}_{K/\mathbb{Q}}(Q(D_p(t))/p)$. This norm is nonzero, because $D_p(t) = \beta^p + \beta^{-p}$ is not a conjugate of t: the only conjugate of t exceeding 2 is t itself, and $\beta^p + \beta^{-p} > t$. Now apply Theorem 5.1. An algebraic integer has norm ± 1 exactly when it is a unit. $\square$

The primes dividing A_p admit a simple description.

Proposition 5.3 (coincidence primes)**.** *In the setting of Corollary 5.2, let $\ell \neq p$ be a prime and $\bar{P}$ the reduction of P modulo ℓ. Then $\ell \mid A_p$ if and only if there is $\rho \in \overline{\mathbb{F}}_\ell$ with $\bar{P}(\rho) = \bar{P}(\rho^p) = 0$.*

Proof. Since $\ell \neq p$, $\ell \mid A_p$ if and only if $\ell \mid \operatorname{Res}(P(x), P(x^p))$. Both polynomials are monic, so the resultant reduces modulo ℓ to $\operatorname{Res}(\bar{P}(x), \bar{P}(x^p))$. This vanishes if and only if the two reductions have a common root in $\overline{\mathbb{F}}_\ell$. $\square$

Corollary 5.4. *If the cyclotomic polynomial Φ_N divides $\bar{P}$ in $\mathbb{F}_\ell[x]$ for some N prime to ℓ, then $\ell \mid A_p$ for every prime $p \nmid \ell N$.*

Proof. The roots of Φ_N in $\overline{\mathbb{F}}_\ell$ are the primitive N-th roots of unity, and $\rho \mapsto \rho^p$ permutes them when $\gcd(p, N) = 1$. $\square$

Corollary 5.5. *Let $\rho \in \overline{\mathbb{F}}_\ell$ be a root of $\bar{P}$ of multiplicative order N. If $p \equiv \pm \ell^j \pmod N$ for some $j \geq 0$, then $\ell \mid A_p$.*

Proof. Then $\rho^p = (\rho^{\ell^j})^{\pm 1}$. Here ρ^{ℓ^j} is a Frobenius conjugate of ρ, and its inverse is again a root because $\bar{P}$ is reciprocal. $\square$

Example 5.6 (Lehmer's polynomial)**.** Modulo 3,

$$L(x) \equiv (x^2 + 1)(x^8 + x^7 - x^6 + x^5 + x^3 - x^2 + x + 1).$$

Equivalently $L(i) = 3i$. By Corollary 5.4 with $N = 4$, $3 \mid A_p$ for every prime $p \geq 5$. On the other hand $A_2 = A_3 = 1$ (Table 5), so

$$\operatorname{Res}(L(x), L(x^2)) = 2^{10}, \qquad \operatorname{Res}(L(x), L(x^3)) = 3^{10}, \qquad \operatorname{Res}(L(x), L(x^5)) = 5^{10} \cdot 3^2,$$

and Dobrowolski's inequality $|R_p| \geq p^d$ is an equality at Lehmer's polynomial for $p = 2, 3$. By Corollary 5.2, $Q_L(D_2(t))/2$ and $Q_L(D_3(t))/3$ are units of $\mathbb{Q}(\lambda+\lambda^{-1})$. By Proposition 5.3, no prime $\ell \neq 2$ admits a root of L modulo ℓ whose square is again a root, and similarly for cubes.

TABLE 5. The cofactor A_p of Corollary 5.2, defined by $|\operatorname{Res}(P(x), P(x^p))| = p^d A_p^2$, for the 23 Salem numbers $\alpha < 1.3$ of degree $d \leq 20$, computed exactly with integer matrices (Appendix D). The identity of Theorem 5.1 and the divisibility $p^{d/2} \mid \operatorname{Res}(Q, Q \circ D_p)$ were verified in all 138 cases.

α	d	A_2	A_3	A_5	A_7	A_{11}	A_{13}
1.176280818	10	1	1	3	3	9	9
1.188368148	18	1	1	1	1	13	1
1.200026524	14	1	1	1	5	5	5
1.202616744	14	1	1	1	1	1	29
1.216391661	10	1	1	1	1	1	11
1.219720859	18	5	1	1	5	5	5
1.230391434	10	1	5	1	5	25	5
1.232613549	20	1	11	11	11	1	94 589
1.236317932	16	1	1	9	9	9	261
1.240726424	12	1	7	7	1	91	49
1.252775937	18	5	5	1	5	25	125
1.253330650	20	1	16	16	32	304	1 328
1.255093517	14	1	5	1	5	2 005	20 005
1.256221154	18	23	1	8	136	16	165 328
1.261230961	10	2	1	8	8	16	256
1.267296443	14	9	1	3	27	81	81
1.280638156	8	1	1	9	9	9	144
1.282495561	20	1	19	19	1 273	19	37 411
1.284616551	18	5	5	1	85	39 275	31 715
1.286395967	18	1	41	41	41	75 809	29 889
1.292039106	20	9	1	9	9	279	153 837
1.293485953	10	1	1	3	15	45	45
1.295675372	18	1	1	8	8	152	28 072

Remark 5.7. Table 5 shows that at the smallest Salem numbers the arithmetic lower bound p^d is attained, or nearly attained, for small primes. For the basic inequality $|R_p| \geq p^d$ that underlies Dobrowolski's method, the loss at these numbers therefore lies entirely in the archimedean upper bound for $|R_p|$. A better arithmetic lower bound alone cannot help at these numbers. Proposition 5.3 explains the recurring small prime factors of A_p in the table: they come from roots of unity, or Frobenius classes, among the roots of P modulo ℓ.

As a consistency check of Proposition 5.3, for each of the 138 pairs (α, p) in Table 5 and every prime $\ell < 60$ with $\ell \neq p$, we compared the divisibility $\ell \mid A_p$ with the existence of a common factor of $\bar{P}(x)$ and $\bar{P}(x^p)$ over $\mathbb{F}_\ell$. All 2208 comparisons agree (Appendix F).

6. Two obstructions to transfer

Two recent results on neighbouring problems used mechanisms that, by the observations below, cannot by themselves address Lehmer's problem. For a polynomial P of degree d with roots $\alpha_1, \ldots, \alpha_d$, let $\mu_P = \frac{1}{d}\sum_i \delta_{\alpha_i}$, and let ν be the uniform probability measure on the unit circle.

6.1. **Distribution-level certificates.** Smith settled the Schur–Siegel–Smyth trace problem by characterizing the weak-$*$ limits of conjugate distributions [20]. A lower bound for $\log M(P)$ obtained from a functional of μ_P that is continuous at ν cannot prove Lehmer's conjecture.

Proposition 6.1. *Let F be a real-valued function on Borel probability measures on $\mathbb{C}$, continuous at ν for the weak-$*$ topology, and suppose $\log M(P) \geq F(\mu_P)$ for every monic $P \in \mathbb{Z}[x]$ all of whose roots are roots of unity. Then*

$$\inf\{F(\mu_P) : P \in \mathbb{Z}[x] \text{ monic, with a root that is not a root of unity, } M(P) = \lambda\} \leq 0.$$

Proof. For $P_n = x^n - 1$ we have $\mu_{P_n} \to \nu$ and $\log M(P_n) = 0$, so $F(\nu) \leq 0$. For $L_n(x) = L(x^n)$ we have $M(L_n) = M(L) = \lambda$, since the roots of L_n are the n-th roots of the roots of L, and $\mu_{L_n} \to \nu$: the roots of L_n have moduli $|\alpha|^{1/n} \to 1$, and their angles are equidistributed in the limit. Hence $F(\mu_{L_n}) \to F(\nu) \leq 0$. □

Numerically, the root angles of $L(x^n)$ have star discrepancy exactly $1/(5n)$ for $n = 1, 4, 16, 64, 256$ (Appendix F), while the measure stays equal to λ. By Bilu's theorem [3], every sequence of algebraic numbers of bounded Mahler measure and degree tending to infinity, which is the regime of Lehmer's problem, has conjugate distributions converging to ν. Any certificate for Lehmer's conjecture must therefore use discontinuous, arithmetic information about finite degrees.

6.2. **Capacity of stars.** Dimitrov proved the Schinzel–Zassenhaus conjecture with a criterion based on logarithmic capacity [9]. The house of an algebraic integer is the maximum modulus of its conjugates. The following computation shows that capacity is sensitive to the house rather than to the Mahler measure.

Proposition 6.2. *For $\rho > 0$ and $d \geq 1$ let $S_{d,\rho} = \{z \in \mathbb{C} : z^d \in [0, \rho^d]\}$ be the symmetric star with d rays of length ρ. Then $\operatorname{cap}(S_{d,\rho}) = \rho\, 4^{-1/d}$.*

Proof. $S_{d,\rho}$ is the preimage of the segment $[0, \rho^d]$, of capacity $\rho^d/4$, under the monic polynomial z^d. For a monic polynomial q of degree d one has $\operatorname{cap}(q^{-1}(E)) = \operatorname{cap}(E)^{1/d}$ [17, Sec. 5.2]. □

Corollary 6.3. $1 - \operatorname{cap}(S_{d,1}) = \frac{\log 4}{d} + O(d^{-2})$.

Numerically, $d(1 - \operatorname{cap}(S_{d,1}))$ equals 1.2945, 1.3647 and 1.3853 for $d = 10, 44, 1000$; the limit is $\log 4 = 1.3863$. For an algebraic integer whose conjugates have moduli at most ρ, a star of radius ρ is the natural region carrying the conjugates. The margin by which the capacity of a radius-one star falls below the threshold 1 is about $(\log 4)/d$, independently of how the product of the large conjugates, which is the Mahler measure, is distributed. A capacity criterion of this kind sees the house, not the measure. Any adaptation to Lehmer's problem would need a weighted or measure-sensitive replacement. Compare the remarks on capacity normalization in [24, manuscript 001].

7. Independent re-verifications

Proposition 7.1. *Lehmer's number is a Parry number:*

$$d_\lambda(1) = 1\,\big(0^{10}\,1\,0^{18}\,1\,0^{12}\,1\,0^{18}\,1\,0^{12}\big)^\infty,$$

so $\mu = 1$ and $\lambda = 74$.

Proof. The engine returns $(\mu, \lambda) = (1, 74)$, and a direct 600-digit simulation reproduces the first 400 digits. □

Using the search program of Appendix E, we enumerated all monic reciprocal integer polynomials of even degree $d \leq 22$ satisfying the necessary power-sum and coefficient bounds for Mahler measure at most 1.17629, and those of degree at most 20 for the bound 1.3005. The survivors were then

classified: exact factorization, and root location by 40-digit arithmetic. For $d = 22$ an amplification step (six Graeffe root-squarings) separates cyclotomic from non-cyclotomic content using Voutier's bound. The non-cyclotomic irreducible factors of measure in $(1, 1.17629]$ are $L(\pm x)$ and $L(\pm x^2)$ only. The Salem numbers below 1.3 of degree at most 20 are exactly the 23 numbers listed in Table 5. These results agree with [15] and with Mossinghoff's list, which extend much further. We report them only as independent numerical confirmations; the root location step was not interval-certified.

Finally, a scan of the 19 682 nonzero bivariate polynomials of bidegree at most $(2, 2)$ with coefficients in $\{-1, 0, 1\}$, followed by high-precision quadrature, found no measure in $(0, \log \lambda)$. The smallest positive measure found is

$$m(1 + x + y + xy + x^2y + xy^2 + x^2y^2) = 0.227481223012351\ldots, \qquad M = 1.25543386626661\ldots,$$

the smallest known limit point of Mahler measures [4, 7]. The scan discarded grid values below 0.02 as cyclotomic-type artifacts without certification, so this is an observation, not a theorem.

8. Limitations

The results of Section 4 are computer-assisted. Their correctness rests on the programs in the appendices, on the error analysis in Proposition 3.2, and on the hardware. The independent checker reduces, but does not eliminate, the risk of shared mistakes. The floating-point bounds use generous constants but have not been formally verified. The cases of trace at most 10 remain open, including $(-3, -1, -7)$, which was once considered a potential counterexample to Schmidt's conjecture [22].

Appendix A. The engine

Arguments are $c_{d-1}, \ldots, c_0$ (the coefficients below the leading one), followed by the iteration budget. Compile with

```
gcc -O3 -march=native betacycle.c -lquadmath -lm
```

```c
/* betacycle.c -- certified orbit of 1 under the beta-transformation, beta Salem (or Pisot).
 * State r_n in Z[beta] (power basis, int64), r_0 = 1, r_{n+1} = beta*r_n - floor(beta*r_n).
 * Digit decision: double-precision FMA Horner with an a priori error bound; if inconclusive,
 * extended precision, then __float128; if still inconclusive: exact zero test, else the norm
 * bound |z(beta)| >= ||z||_1^{-(d-1)} (all non-identity conjugates of beta have modulus <= 1).
 * Cycle detection: Brent + distinguished points (low q bits of r0^(r1<<7)^(r2<<13), q=14).
 * If unresolved: index N0 of the last strict record of max_i |r_i| gives mu+lambda > N0.
 * Usage: betacycle c_{d-1} ... c_0 budget [qBITS]   (P = x^d + c_{d-1}x^{d-1} + ... + c_0)
 * Identical to the program used for the campaign except for comments and the MAXD guard;
 * compile with -DMAXD=16 for degrees 9..16.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <stdint.h>
#include <quadmath.h>
typedef long long ll;
#ifndef MAXD
#define MAXD 8
#endif
static int d; static ll c[MAXD];
static long double B1, EK; static double Bd, EKd; static __float128 betaq; static double bpd[MAXD];
static ll fb1=0, fb2=0, fbz=0;

static __float128 Peval(__float128 x){ __float128 v=1.0Q; for(int i=d-1;i>=0;i--) v=v*x+(__float128)c[i];
    return v; }
static void compute_beta(void){
  __float128 lo=1.0Q, hi=2.0Q; for(int i=0;i<d;i++) hi+=fabsq((__float128)c[i]);
  if(Peval(lo)>=0){ fprintf(stderr,"P(1)>=0\n"); exit(2); }
  for(int it=0;it<400;it++){ __float128 m=(lo+hi)/2; if(Peval(m)<0) lo=m; else hi=m; }
  betaq=(lo+hi)/2; B1=(long double)betaq;
```

```
  double S=0, p=1; for(int i=0;i<d;i++){ bpd[i]=p; S+=p; p*=(double)betaq; }
  EK = (long double)(8.0*(d+2)*5.421010862427522e-20*S*1.0001);
  Bd=(double)betaq; EKd = 8.0*(d+2)*1.1102230246251565e-16*S*1.0001;
}
static ll decide_slow(const ll *y){
  fb1++;
  double scale=0; for(int i=0;i<d;i++) scale+=fabs((double)y[i])*bpd[i];
  __float128 vq=0; for(int i=d-1;i>=0;i--) vq=vq*betaq+(__float128)y[i];
  __float128 Eq=(__float128)scale*64.0Q*(d+2)*1.0e-33Q;
  __float128 fq=floorq(vq);
  if(vq-fq>Eq && fq+1-vq>Eq) return (ll)fq;
  fb2++;
  ll ks=(ll)roundq(vq); int allzero=(y[0]-ks==0); double l1=fabs((double)(y[0]-ks));
  for(int i=1;i<d;i++){ if(y[i]) allzero=0; l1+=fabs((double)y[i]); }
  if(allzero){ fbz++; return ks; }
  if(log((double)Eq) < -(d-1)*log(l1) - 2.0) return (vq-(__float128)ks>0)? ks : ks-1;
  fprintf(stderr,"UNDECIDABLE digit; abort\n"); exit(3);
}
static inline ll step(ll *r, ll *rmax_out){
  ll top=r[d-1]; ll y[MAXD]; ll ym=0;
  y[0]=-top*c[0];
  for(int i=1;i<d;i++) y[i]=r[i-1]-top*c[i];
  for(int i=0;i<d;i++){ ll a=y[i]<0?-y[i]:y[i]; if(a>ym) ym=a; }
  double vd=(double)y[d-1];
  for(int i=d-2;i>=0;i--) vd=__builtin_fma(vd,Bd,(double)y[i]);
  double Ed=(double)ym*EKd;
  ll k=(ll)vd; if(vd<0) k=(ll)floor(vd);
  double frd=vd-(double)k;
  if(__builtin_expect(!(frd>Ed && frd<1.0-Ed),0)){
    long double v=(long double)y[d-1];
    for(int i=d-2;i>=0;i--) v=v*B1+(long double)y[i];
    long double E=(long double)ym*EK;
    k=(ll)v; if(v<0) k=(ll)floorl(v);
    long double fr=v-(long double)k;
    if(!(fr>E && fr<1.0L-E)) k=decide_slow(y);
  }
  y[0]-=k;
  ll rm=ym; ll a0=y[0]<0?-y[0]:y[0]; if(a0>rm) rm=a0;   /* upper estimate; exact max computed below if
     needed */
  memcpy(r,y,sizeof(ll)*d);
  *rmax_out=rm;
  return k;
}
static inline int eq(const ll*a,const ll*b){ for(int i=0;i<d;i++) if(a[i]!=b[i]) return 0; return 1; }
static inline uint64_t hsh(const ll*a){
  static const uint64_t K[MAXD]={0x9E3779B97F4A7C15ULL,0xC2B2AE3D27D4EB4FULL,0x165667B19E3779F9ULL,0
     xD6E8FEB86659FD93ULL,
     0xFF51AFD7ED558CCDULL,0xC4CEB9FE1A85EC53ULL,0x94D049BB133111EBULL,0xBF58476D1CE4E5B9ULL};
  uint64_t h=0; for(int i=0;i<d;i++) h+=(uint64_t)a[i]*K[i]; h^=h>>29; h*=0xBF58476D1CE4E5B9ULL; h^=h>>32;
      return h; }
typedef struct { ll s[MAXD]; ll idx; int used; } Ent;
static Ent *tab; static size_t tabsz;
static Ent* lookup_insert(const ll*s, ll idx, int*found){
  size_t pos=(size_t)(hsh(s)*0x9E3779B97F4A7C15ULL>>20)&(tabsz-1);
  while(tab[pos].used){ if(eq(tab[pos].s,s)){*found=1; return &tab[pos];} pos=(pos+1)&(tabsz-1); }
  tab[pos].used=1; memcpy(tab[pos].s,s,sizeof(ll)*d); tab[pos].idx=idx; *found=0; return &tab[pos]; }
static ll exact_max(const ll*r){ ll m=0; for(int i=0;i<d;i++){ ll a=r[i]<0?-r[i]:r[i]; if(a>m) m=a; }
     return m; }

int main(int argc,char**argv){
  if(argc<4){ fprintf(stderr,"usage\n"); return 1; }
  int nargs=argc-1, q=14; if(argv[argc-1][0]=='q'){ q=atoi(argv[argc-1]+1); nargs--; }
  ll budget=atoll(argv[nargs]); d=nargs-1; if(d>MAXD){ fprintf(stderr,"degree %d exceeds MAXD\n",d);
     return 5; }
  for(int i=0;i<d;i++) c[d-1-i]=atoll(argv[1+i]);
```

```
  compute_beta(); char bs[64]; quadmath_snprintf(bs,64,"%.22Qg",betaq);
  tabsz=1u<<22; tab=calloc(tabsz,sizeof(Ent)); if(!tab) return 1;
  uint64_t dpmask=(1ULL<<q)-1;
  ll x0[MAXD]={0}; x0[0]=1;
  ll tort[MAXD], hare[MAXD]; memcpy(tort,x0,sizeof(ll)*d); memcpy(hare,x0,sizeof(ll)*d);
  ll power=1, lam=0, n=0, tidx=0, fi=-1, fj=-1; int method=0;
  ll best=exact_max(x0), recidx=0, rm;
  while(n<budget){
    step(hare,&rm); n++; lam++;
    if(rm>=best){ ll em=exact_max(hare); if(em>best){ best=em; recidx=n; } }
    if(eq(hare,tort)){ fi=tidx; fj=n; method=1; break; }
    if((( (uint64_t)hare[0] ^ ((uint64_t)hare[1]<<7) ^ ((uint64_t)hare[2]<<13) ) & dpmask)==0){ int f; Ent
     *e=lookup_insert(hare,n,&f); if(f){ fi=e->idx; fj=n; method=2; break; } }
    if(lam==power){ memcpy(tort,hare,sizeof(ll)*d); tidx=n; power<<=1; lam=0; }
    if(__builtin_expect((n&0xFFFFFF)==0,0)){ if(best>(1LL<<50)){ fprintf(stderr,"overflow risk\n"); return
      4; } }
  }
  if(fi<0){
    int K=0; while(((1LL<<(K+1))-1)+(1LL<<(K+1))<=n) K++;
    printf("UNRESOLVED beta=%s steps=%lld record_bound=%lld brent_bound=%lld maxcoord=%lld fb=%lld/%lld/%
     lld\n",
           bs,n,recidx,(1LL<<K),best,fb1,fb2,fbz);
    return 0;
  }
  ll s[MAXD]; memcpy(s,hare,sizeof(ll)*d); ll w[MAXD]; memcpy(w,s,sizeof(ll)*d); ll lamx=0;
  do{ step(w,&rm); lamx++; }while(!eq(w,s));
  /* mu: start from the latest stored DP (index < fi) that is NOT on the cycle, else from x0 */
  ll a[MAXD]; memcpy(a,x0,sizeof(ll)*d); ll start=0;
  { size_t m=0; ll *ids=malloc(sizeof(ll)*tabsz); size_t *ps=malloc(sizeof(size_t)*tabsz);
    for(size_t p=0;p<tabsz;p++) if(tab[p].used && tab[p].idx<fi){ ids[m]=tab[p].idx; ps[m]=p; m++; }
    for(int tries=0; tries<64 && m>0; tries++){
      size_t bi=0; for(size_t t=1;t<m;t++) if(ids[t]>ids[bi]) bi=t;
      ll tmp[MAXD]; memcpy(tmp,tab[ps[bi]].s,sizeof(ll)*d); ll w2[MAXD]; memcpy(w2,tmp,sizeof(ll)*d);
      for(ll u=0;u<lamx;u++) step(w2,&rm);
      if(!eq(w2,tmp)){ memcpy(a,tmp,sizeof(ll)*d); start=ids[bi]; break; }
      ids[bi]=ids[m-1]; ps[bi]=ps[m-1]; m--;
    }
    free(ids); free(ps); }
  ll b[MAXD]; memcpy(b,a,sizeof(ll)*d); for(ll u=0;u<lamx;u++) step(b,&rm);
  ll mu=start; while(!eq(a,b)){ step(a,&rm); step(b,&rm); mu++; }
  printf("RESOLVED beta=%s mu=%lld lambda=%lld detect=%s@%lld maxcoord=%lld fb=%lld/%lld/%lld\n",
         bs,mu,lamx,method==1?"Brent":"DP",n,best,fb1,fb2,fbz);
  return 0;
}
```

Appendix B. The independent checker

Arguments are $c_{d-1}, \ldots, c_0$, then μ and λ.

```
/* betacheck.c -- independent certificate checker for an eventually periodic beta-expansion.
 * Given P (sextic or any degree <= 16), MU and LAMBDA, verifies by direct iteration from r_0 = 1:
 *   (1) r_{MU} = r_{MU+LAMBDA}                      (eventual periodicity)
 *   (2) r_{MU+j} != r_{MU} for 0 < j < LAMBDA        (LAMBDA is the exact period)
 *   (3) r_{MU-1} != r_{MU-1+LAMBDA}                  (MU is the exact preperiod)
 * Digit decisions: long double Horner with a priori bound, __float128 fallback, exact fallback
 * (zero test + norm bound). Independent of betacycle4 (no Brent/DP, different primary precision).
 * Also prints a checksum of the digit sequence d_1..d_{MU+LAMBDA}.
 * Usage: betacheck c_{d-1} ... c_0 MU LAMBDA
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <stdint.h>
```

```c
#include <quadmath.h>
typedef long long ll;
#define MAXD 16
static int d; static ll c[MAXD]; static long double B1, EK; static __float128 bq; static double bpd[MAXD];
static ll nf1=0,nf2=0;
static __float128 Pev(__float128 x){ __float128 v=1; for(int i=d-1;i>=0;i--) v=v*x+(__float128)c[i];
    return v; }
static ll slow(const ll*y){
  nf1++; double sc=0; for(int i=0;i<d;i++) sc+=fabs((double)y[i])*bpd[i];
  __float128 v=0; for(int i=d-1;i>=0;i--) v=v*bq+(__float128)y[i];
  __float128 E=(__float128)sc*64.0Q*(d+2)*1.0e-33Q, f=floorq(v);
  if(v-f>E && f+1-v>E) return (ll)f;
  nf2++; ll ks=(ll)roundq(v); int z=(y[0]==ks); double l1=fabs((double)(y[0]-ks));
  for(int i=1;i<d;i++){ if(y[i]) z=0; l1+=fabs((double)y[i]); }
  if(z) return ks;
  if(log((double)E) < -(d-1)*log(l1)-2) return (v-(__float128)ks>0)?ks:ks-1;
  fprintf(stderr,"undecidable\n"); exit(3);
}
static inline ll step(ll*r){
  ll t=r[d-1], y[MAXD]; y[0]=-t*c[0]; for(int i=1;i<d;i++) y[i]=r[i-1]-t*c[i];
  ll ym=0; for(int i=0;i<d;i++){ ll a=llabs(y[i]); if(a>ym) ym=a; }
  long double v=(long double)y[d-1]; for(int i=d-2;i>=0;i--) v=v*B1+(long double)y[i];
  long double E=(long double)ym*EK, f=floorl(v); ll k;
  if(v-f>E && f+1-v>E) k=(ll)f; else k=slow(y);
  y[0]-=k; memcpy(r,y,sizeof(ll)*d); return k;
}
static int eq(const ll*a,const ll*b){ for(int i=0;i<d;i++) if(a[i]!=b[i]) return 0; return 1; }
int main(int argc,char**argv){
  int n=argc-1; ll MU=atoll(argv[n-1]), LA=atoll(argv[n]); d=n-2;
  for(int i=0;i<d;i++) c[d-1-i]=atoll(argv[1+i]);
  __float128 lo=1,hi=2; for(int i=0;i<d;i++) hi+=fabsq((__float128)c[i]);
  for(int it=0;it<400;it++){ __float128 m=(lo+hi)/2; if(Pev(m)<0) lo=m; else hi=m; }
  bq=(lo+hi)/2; B1=(long double)bq; double S=0,p=1; for(int i=0;i<d;i++){ bpd[i]=p; S+=p; p*=(double)bq; }
  EK=(long double)(16.0*(d+2)*5.421010862427522e-20*S);
  ll r[MAXD]={0}; r[0]=1; uint64_t chk=14695981039346656O3ULL; ll k;
  for(ll i=1;i<=MU-1;i++){ k=step(r); chk=(chk^(uint64_t)k)*1099511628211ULL; }
  ll before[MAXD]; memcpy(before,r,sizeof(ll)*d);              /* r_{MU-1} */
  k=step(r); chk=(chk^(uint64_t)k)*1099511628211ULL;
  ll at[MAXD]; memcpy(at,r,sizeof(ll)*d);                      /* r_{MU} */
  int early=0, prem=0;
  for(ll j=1;j<=LA;j++){
    if(j==LA){ /* r currently r_{MU+LA-1}: compare with r_{MU-1} */ if(eq(r,before)) prem=1; }
    k=step(r); chk=(chk^(uint64_t)k)*1099511628211ULL;
    if(j<LA && eq(r,at)) { early=1; printf("early return at j=%lld\n",j); break; }
  }
  int per=eq(r,at);
  printf("check: periodic(r_MU==r_MU+LA)=%s exact_period=%s exact_preperiod=%s digit_checksum=%016llx
     fallbacks=%lld/%lld\n",
         per?"YES":"NO", (!early)?"YES":"NO", (!prem)?"YES":"NO", (unsigned long long)chk, nf1, nf2);
  return !(per && !early && !prem);
}
```

Appendix C. Campaign driver and validation script

The driver below produced Tables 3 and 4; the second script produced Table 2.

```python
import subprocess, time, sys, os
cases = []
for line in open('open64.txt'):
    a,b,c,bd = [int(v) for v in line.split()]
    cases.append((bd, a, b, c))
cases.sort()
queue = []
for bd,a,b,c in cases:
```

```
    if -a <= 10: continue                    # trace <= 10: Stockton bounds 3e10-9e10, beyond budget
    if bd <= 1.1e9: budget = 6_000_000_000
    elif bd <= 3.2e9: budget = 15_000_000_000
    else: budget = 20_000_000_000
    queue.append((bd, a, b, c, budget))
log = open('campaign_log.txt', 'a')
def launch(item):
    bd,a,b,c,budget = item
    cmd = ['./betacycle4', str(a), str(b), str(c), str(b), str(a), '1', str(budget)]
    return subprocess.Popen(cmd, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True), item, time.time()
running = []
qi = 0
while qi < len(queue) or running:
    while len(running) < 2 and qi < len(queue):
        running.append(launch(queue[qi])); qi += 1
    time.sleep(2)
    still = []
    for p, item, t0 in running:
        if p.poll() is None: still.append((p, item, t0)); continue
        out = p.stdout.read().strip()
        bd,a,b,c,budget = item
        log.write(f"{time.strftime('%H:%M:%S')} ({a},{b},{c}) trace={-a} stockton_bound={bd} budget={budget} secs={time.time()-t0:.0f} :: {out}\n")
        log.flush()
    running = still
log.write("CAMPAIGN DONE\n"); log.flush()
```

```
"""Recompute resolved entries of Stockton's tables with the paper's engine (Appendix A)
and write validation_log.txt (one line per case). Two cases run in parallel."""
import subprocess, re
CASES = [  # (a, b, c, m, p) from Stockton, arXiv:1912.10386, Tables 1 and 3
 (-7,-29,-43,1039779,90), (-11,-11,-26,1285570,677), (-14,-36,-45,2098011,112),
 (-14,13,-29,1428555,7640), (-11,-14,-28,1490333,72458), (-13,-6,-27,10913948,13914931),
 (-15,-43,-62,11994574,217750), (-12,-47,-69,18789419,12521), (-11,-35,-49,48516722,3128603),
 (-13,-30,-41,137293807,164656), (-7,-28,-41,159781799,94829), (-13,-38,-55,191196227,16067),
 (-12,-43,-62,779478947,96687), (-8,-30,-44,2121493281,188611456), (-9,-35,-51,4075324464,425719617),
]
def run(case):
    a,b,c,m,p = case
    budget = str(3*(m+p) + 10**7)
    return subprocess.Popen(['./betacycle', str(a), str(b), str(c), str(b), str(a), '1', budget],
                            stdout=subprocess.PIPE, text=True), case
out = open('validation_log.txt', 'w')
pending = list(CASES); running = []
while pending or running:
    while len(running) < 2 and pending:
        running.append(run(pending.pop(0)))
    proc, case = running.pop(0)
    txt = proc.communicate()[0]
    a,b,c,m,p = case
    mu = int(re.search(r'mu=(\d+)', txt).group(1)); la = int(re.search(r'lambda=(\d+)', txt).group(1))
    out.write(f"VAL ({a},{b},{c}) stockton={m},{p} ours={mu},{la} {'MATCH' if (mu,la)==(m,p) else 'MISMATCH'}\n"); out.flush()
print(open('validation_log.txt').read())
```

Appendix D. Exact verification of Theorem 5.1 and Table 5

The script computes $\mathrm{Res}(P(x), P(x^p)) = \det P(C^p)$ and $\mathrm{Res}(Q, Q \circ D_p) = \det Q(D_p(C_Q))$ with integer companion matrices, so no root of any polynomial is computed.

```
"""Exact verification of the resultant square law and the A_p table (integer matrices only).
Res(P(x), P(x^p)) = det P(C^p),  C = companion matrix of monic P.
```

```
Res(Q, Q o D_p)   = det Q(D_p(C_Q)), C_Q = companion matrix of the trace polynomial Q.
Claim: det P(C^p) = (det Q(D_p(C_Q)))^2  and  p^m | det Q(D_p(C_Q)),  A_p = |det Q(D_p(C_Q))| / p^m.
"""
from sympy import Matrix, eye, zeros, primerange
S = [
 (1.176280818260,[1,1,0,-1,-1,-1,-1,-1,0,1,1]),
 (1.188368147508,[1,-1,1,-1,0,0,-1,1,-1,1,-1,1,-1,0,0,-1,1,-1,1]),
 (1.200026523987,[1,0,0,-1,-1,0,0,1,0,0,-1,-1,0,0,1]),
 (1.202616743689,[1,0,-1,0,0,0,0,-1,0,0,0,0,-1,0,1]),
 (1.216391661138,[1,0,0,0,-1,-1,-1,0,0,0,1]),
 (1.219720859040,[1,-1,0,0,0,0,0,0,-1,1,-1,0,0,0,0,0,0,-1,1]),
 (1.230391434407,[1,0,0,-1,0,-1,0,-1,0,0,1]),
 (1.232613548593,[1,-1,0,0,0,-1,1,0,0,-1,1,-1,0,0,1,-1,0,0,0,-1,1]),
 (1.236317931803,[1,-1,0,0,0,0,0,0,0,-1,0,0,0,0,0,0,0,-1,1]),
 (1.240726423653,[1,-1,1,-1,0,0,-1,0,0,-1,1,-1,1]),
 (1.252775937410,[1,0,0,0,0,0,0,-1,-1,-1,-1,-1,-1,-1,0,0,0,0,0,1]),
 (1.253330650201,[1,0,-1,0,0,-1,0,0,0,0,0,0,0,0,0,-1,0,0,-1,0,1]),
 (1.255093516764,[1,0,-1,-1,0,1,0,-1,0,1,0,-1,-1,0,1]),
 (1.256221154392,[1,-1,0,0,-1,1,0,0,0,-1,0,0,0,1,-1,0,0,-1,1]),
 (1.261230961137,[1,0,-1,0,0,-1,0,0,-1,0,1]),
 (1.267296442523,[1,-1,0,0,0,0,0,-1,1,-1,0,0,0,0,-1,1]),
 (1.280638156268,[1,0,0,-1,-1,-1,0,0,1]),
 (1.282495560640,[1,-2,2,-2,2,-2,1,0,-1,1,-1,1,-1,0,1,-2,2,-2,2,-2,1]),
 (1.284616550926,[1,0,0,0,-1,0,-1,-1,0,-1,0,-1,-1,0,-1,0,0,0,1]),
 (1.286395966836,[1,-2,2,-2,2,-2,2,-3,3,-3,3,-3,2,-2,2,-2,2,-2,1]),
 (1.292039106018,[1,0,-1,0,0,-1,0,0,-1,0,1,0,-1,0,0,-1,0,0,-1,0,1]),
 (1.293485953125,[1,0,-1,-1,0,1,0,-1,-1,0,1]),
 (1.295675371944,[1,-1,0,0,-1,1,-1,0,1,-1,1,0,-1,1,-1,0,0,-1,1]),
]
def companion(coeffs):           # coeffs highest-first, monic
    n = len(coeffs)-1; C = zeros(n, n)
    for i in range(1, n): C[i, i-1] = 1
    for i in range(n): C[i, n-1] = -coeffs[n-i]
    return C
def poly_at_matrix(coeffs, M):    # Horner, coeffs highest-first
    R = zeros(M.shape[0], M.shape[0])
    for a in coeffs: R = R*M + a*eye(M.shape[0])
    return R
def trace_poly(c):               # P(x) = x^m Q(x+1/x), Q highest-first
    d = len(c)-1; m = d//2
    D = [[2], [1, 0]]            # Dickson D_0 = 2, D_1 = u (highest-first lists)
    def sub(a, b):
        n = max(len(a), len(b)); a = [0]*(n-len(a))+a; b = [0]*(n-len(b))+b; return [x-y for x,y in zip(a,
     b)]
    for k in range(2, m+1): D.append(sub(D[-1]+[0], D[-2]))
    Q = [0]*(m+1); Q[-1] += c[m]
    for k in range(1, m+1):
        Dk = D[k]; Dk = [0]*(m+1-len(Dk)) + Dk
        Q = [q + c[m-k]*x for q, x in zip(Q, Dk)]
    return Q, m
def dickson_matrix(p, M):        # D_p(M) via recurrence D_k = M D_{k-1} - D_{k-2}
    I = eye(M.shape[0]); A, B = 2*I, M
    if p == 0: return A
    for _ in range(p-1): A, B = B, M*B - A
    return B
if __name__ == "__main__":
    primes = list(primerange(2, 14))
    allok = True
    rows = []
    for a, c in S:
        Q, m = trace_poly(c)
        C, CQ = companion(c), companion(Q)
        Ap = []
        for p in primes:
            RP = poly_at_matrix(c, C**p).det()
```

```
            RQ = poly_at_matrix(Q, dickson_matrix(p, CQ)).det()
            ok = (RP == RQ**2) and (RQ % p**m == 0)
            allok &= ok
            Ap.append(int(abs(RQ)//p**m))
        rows.append((a, len(c)-1, Ap))
    print("square law & divisibility verified exactly for all 23 x", len(primes), "primes:", allok)
    print("alpha d " + " ".join(f"A_{p}" for p in primes))
    for a, dd, Ap in rows:
        print(f"ROW {a:.9f} {dd} " + " ".join(str(v) for v in Ap))
```

Appendix E. Search program for small Mahler measures

```
/* dfs.c -- exhaustive enumeration of monic palindromic integer polynomials of even degree d
 * with Mahler measure <= Mb, by depth-first search on power sums (Newton identities).
 * Necessary conditions used (all rigorous): |p_n| <= Mb^n + d - 1 for all n, and
 * |a_k| <= C(d-1,k) Mb + C(d-1,k-1). Survivors are printed (coefficients, highest first)
 * for subsequent exact classification (factorization + root location).
 * Usage: dfs d Mb outfile
 */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
typedef long long ll;
static int d, m, EXT; static double Mb;
static ll B[128], bnd_a[64], a[64], p[128];
static ll leaves=0, surv=0, internal=0; static FILE *out;
static double binom(int n,int k){ double r=1; for(int i=1;i<=k;i++) r=r*(n-k+i)/i; return r; }
static void leaf(void){
  leaves++; ll c[128]; c[0]=1;
  for(int i=1;i<=m;i++) c[i]=a[i];
  for(int j=1;j<=m;j++) c[m+j]=a[m-j];
  ll pn[128]; for(int i=1;i<=m;i++) pn[i]=p[i];
  for(int n=m+1;n<=EXT;n++){
    ll s=(n<=d)?(ll)n*c[n]:0; int imax=n<d?n:d;
    for(int i=1;i<=imax;i++){ int idx=n-i; if(idx>=1&&c[i]) s+=c[i]*pn[idx]; }
    ll v=-s; if(v>B[n]||v<-B[n]) return; pn[n]=v;
  }
  surv++; for(int i=0;i<=d;i++) fprintf(out,"%lld ",c[i]); fprintf(out,"\n");
}
static void rec(int k){
  if(k>m){ leaf(); return; }
  internal++; ll ssum=0; for(int i=1;i<k;i++) ssum+=a[i]*p[k-i];
  ll Bk=B[k], r=((-ssum)%k+k)%k, start=-Bk+(((r-(-Bk))%k+k)%k);
  for(ll pk=start;pk<=Bk;pk+=k){ ll ak=-(pk+ssum)/k; if(ak>bnd_a[k]||ak<-bnd_a[k]) continue; a[k]=ak; p[k
     ]=pk; rec(k+1); }
}
int main(int argc,char**argv){
  d=atoi(argv[1]); Mb=atof(argv[2]); m=d/2; EXT=3*m; out=fopen(argv[3],"w");
  for(int n=1;n<=EXT;n++) B[n]=(ll)floor(pow(Mb,n)+d-1);
  for(int k=1;k<=m;k++) bnd_a[k]=(ll)floor(binom(d-1,k)*Mb+binom(d-1,k-1));
  a[0]=1; rec(1); fclose(out);
  fprintf(stderr,"d=%d Mb=%g internal=%lld leaves=%lld survivors=%lld\n",d,Mb,internal,leaves,surv);
  return 0;
}
```

Appendix F. Consistency checks

```
"""Consistency checks quoted in Sections 5 and 6.
(1) Star discrepancy of the root angles of L(x^n) (Section 6.1).
(2) Proposition 5.3 on the A_p table: for primes l < 60, l != p,
    l | A_p  iff  gcd(P(x), P(x^p)) != 1 over F_l.   Reads squarelaw_out.txt."""
import math
```

```
import numpy as np, mpmath as mp
from sympy import Poly, symbols, primerange
from squarelaw import S
x = symbols('x')
L = [1,1,0,-1,-1,-1,-1,-1,0,1,1]
mp.mp.dps = 30
rootsL = mp.polyroots([mp.mpf(c) for c in L], maxsteps=300, extraprec=200)
for n in [1, 4, 16, 64, 256]:
    angs = [((float(mp.arg(b)) + 2*math.pi*k)/n) % (2*math.pi) for b in rootsL for k in range(n)]
    a = np.sort(np.array(angs))/(2*math.pi); N = len(a); i = np.arange(1, N+1)
    D = max(np.max(i/N - a), np.max(a - (i-1)/N))
    print(f"n={n:4d}: star discrepancy {D:.6f}   1/(5n) = {1/(5*n):.6f}")
rows = [ln.split() for ln in open('squarelaw_out.txt') if ln.startswith('ROW')]
checked = bad = 0
for (al, c), row in zip(S, rows):
    P = Poly(c, x)
    for p, A in zip([2,3,5,7,11,13], [int(v) for v in row[3:]]):
        Pp = Poly(P.as_expr().subs(x, x**p), x)
        for l in primerange(2, 60):
            if l == p: continue
            g = Poly(P.as_expr(), x, modulus=l).gcd(Poly(Pp.as_expr(), x, modulus=l))
            bad += ((g.degree() > 0) != (A % l == 0)); checked += 1
print(f"Proposition 5.3 consistency: {checked} triples checked, {bad} mismatches")
```

References

[1] S. Akiyama and H. Hichri, *Periodic expansion of one by Salem numbers*, Ergodic Theory Dynam. Systems **43** (2023), no. 9, 2841–2862.

[2] S. Akiyama and Y. Tanigawa, *Salem numbers and uniform distribution modulo* 1, Publ. Math. Debrecen **64** (2004), 329–341.

[3] Y. Bilu, *Limit distribution of small points on algebraic tori*, Duke Math. J. **89** (1997), 465–476.

[4] P. Borwein, K. G. Hare and M. J. Mossinghoff, *The Mahler measure of polynomials with odd coefficients*, Bull. London Math. Soc. **36** (2004), 332–338.

[5] D. W. Boyd, *Salem numbers of degree four have periodic expansions*, in: Théorie des nombres (Quebec, 1987), de Gruyter, Berlin, 1989.

[6] D. W. Boyd, *On the beta expansion for Salem numbers of degree* 6, Math. Comp. **65** (1996), no. 214.

[7] D. W. Boyd and M. J. Mossinghoff, *Small limit points of Mahler's measure*, Experiment. Math. **14** (2005), no. 4.

[8] R. P. Brent, *An improved Monte Carlo factorization algorithm*, BIT **20** (1980), 176–184.

[9] V. Dimitrov, *A proof of the Schinzel–Zassenhaus conjecture on polynomials*, arXiv:1912.12545.

[10] E. Dobrowolski, *On a question of Lehmer and the number of irreducible factors of a polynomial*, Acta Arith. **34** (1979), 391–401.

[11] H. Hichri, *On the beta expansion of Salem numbers of degree* 8, LMS J. Comput. Math. **17** (2014).

[12] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002.

[13] D. H. Lehmer, *Factorization of certain cyclotomic functions*, Ann. of Math. (2) **34** (1933), 461–479.

[14] R. Lidl, G. L. Mullen and G. Turnwald, *Dickson Polynomials*, Longman, 1993.

[15] M. J. Mossinghoff, G. Rhin and Q. Wu, *Minimal Mahler measures*, Experiment. Math. **17** (2008), no. 4.

[16] W. Parry, *On the β-expansions of real numbers*, Acta Math. Acad. Sci. Hungar. **11** (1960), 401–416.

[17] T. Ransford, *Potential Theory in the Complex Plane*, London Math. Soc. Student Texts 28, Cambridge Univ. Press, 1995.

[18] A. Rényi, *Representations for real numbers and their ergodic properties*, Acta Math. Acad. Sci. Hungar. **8** (1957), 477–493.

[19] K. Schmidt, *On periodic expansions of Pisot numbers and Salem numbers*, Bull. London Math. Soc. **12** (1980), 269–278.

[20] A. Smith, *Algebraic integers with conjugates in a prescribed distribution*, Ann. of Math. (2) **200** (2024), no. 1; arXiv:2111.12660.

[21] C. Smyth, *Seventy years of Salem numbers*, Bull. London Math. Soc. **47** (2015), 379–395.

[22] J. J. Stockton, *On the co-factors of degree* 6 *Salem number beta expansions*, arXiv:1912.10386.

[23] P. C. van Oorschot and M. J. Wiener, *Parallel collision search with cryptanalytic applications*, J. Cryptology **12** (1999), 1–28.

[24] KEO, *Verifiable Mathematical Reasoning Corpus*, research drafts (2026), https://github.com/QEEDDD/Verifiable-Mathematical-Reasoning-Corpus.
[25] P. Voutier, *An effective lower bound for the height of algebraic numbers*, Acta Arith. **74** (1996), 81–95.